

CHƯƠNG 20: TRI THỨC TRONG HỌC TẬP

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 20

- ◇ 20.1 Biểu diễn Logic của Học tập (Logical Formulation)
- ◇ 20.2 Tri thức trong Học tập (Knowledge in Learning)
- ◇ 20.3 Học tập Dựa trên Giải thích (Explanation-Based Learning)
- ◇ 20.4 Học sử dụng Thông tin Tương quan (Relevance Info)
- ◇ 20.5 Lập trình Logic Quy nạp (Inductive Logic Programming)

20.1 Biểu diễn Logic của Học tập

◇ Học máy truyền thống đi tìm một hàm số toán học (Ví dụ: phương trình đường thẳng, cây quyết định).

◇ **Khung tư duy Logic (Logical Formulation):**

- Học tập là quá trình đi tìm một **Giả thuyết (Hypothesis - h)**.
- h là một tập hợp các mệnh đề logic biểu diễn các quy luật của thế giới.

◇ Một giả thuyết h được coi là *Đúng đắn (Consistent)* nếu nó tương thích với toàn bộ các **Ví dụ (Examples)** trong tập huấn luyện.

- $Hypothesis \wedge Descriptions \models Classifications$

20.1 Tìm kiếm Giả thuyết Tốt nhất (Current-best)

◇ Nếu coi Học tập là một quá trình Tìm kiếm (Search), ta có thuật toán **Current-best-hypothesis search**:

1. Bắt đầu với một giả thuyết ngẫu nhiên.
2. Xét từng ví dụ huấn luyện.
3. Nếu ví dụ là *Dương tính giả (False positive)*: Làm cho giả thuyết **Đặc thù hơn (Specialize)**.
4. Nếu ví dụ là *Âm tính giả (False negative)*: Làm cho giả thuyết **Tổng quát hơn (Generalize)**.

◇ Nhược điểm: Thuật toán có thể bị kẹt ở cực tiểu cục bộ (Local optima) do phải đưa ra quá nhiều lựa chọn không chắc chắn.

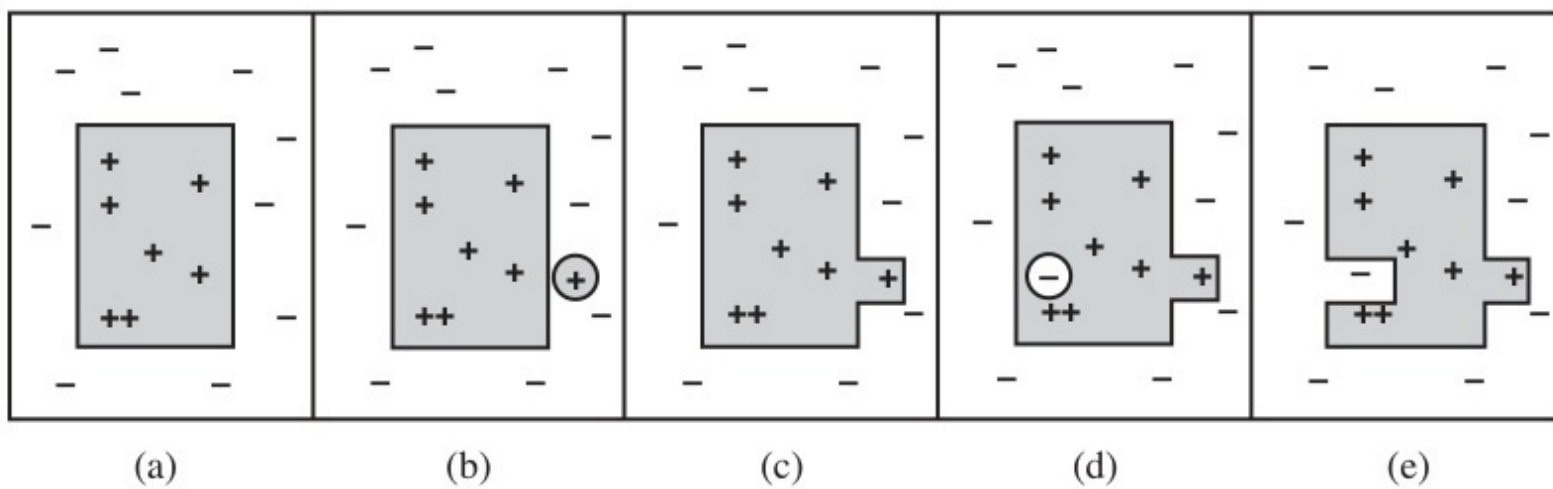


Figure 1: Mô phỏng các trạng thái của giả thuyết: (a) Nhất quán, (b) Âm tính giả, (c) Tổng quát hóa, (d) Dương tính giả.

20.1 Không gian phiên bản (Version Space)

◇ Tìm kiếm Ít cam kết nhất (Least-commitment search):

- Thay vì theo dõi 1 giả thuyết, ta giữ lại **Mọi giả thuyết (Version Space)** thỏa mãn các ví dụ đã thấy.
- Ban đầu Không gian phiên bản cực kỳ rộng lớn. Cứ mỗi lần gặp một ví dụ mới, ta thu hẹp dần tập hợp này lại.

◇ Biểu diễn Giới hạn (Boundary representation):

- Không gian phiên bản được định hình bởi hai đường biên:
- ****G-set (General set):**** Giả thuyết tổng quát nhất còn đúng.
- ****S-set (Specific set):**** Giả thuyết đặc thù nhất còn đúng.

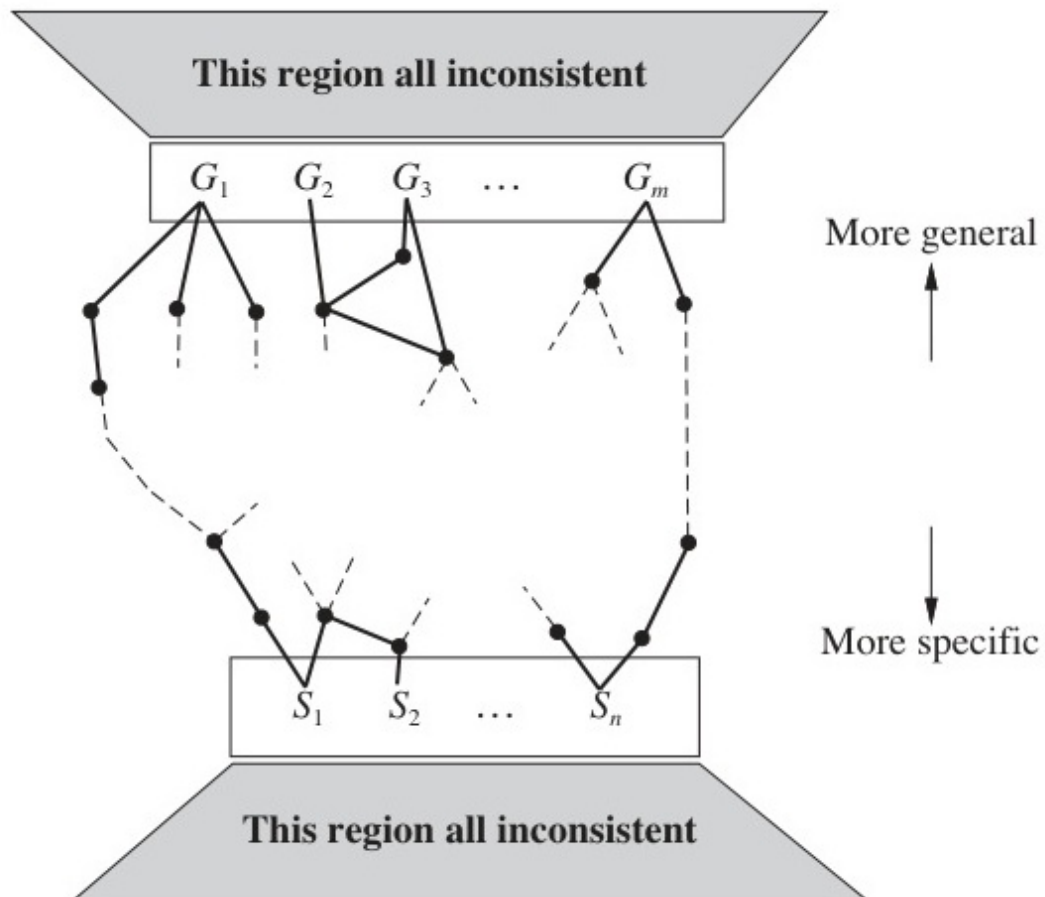


Figure 2: Không gian phiên bản chứa tất cả các giả thuyết nhất quán với các ví dụ, nằm giữa hai tập giới hạn G và S .

20.2 Tri thức trong Học tập

◇ Học máy (Machine Learning) thuần túy cần một lượng dữ liệu vô cùng khổng lồ để nhận ra mẫu quy luật.

◇ **Tại sao con người chỉ cần 1 ví dụ là đã học được?**

- Vì con người có **Tri thức nền (Background Knowledge)**.
- Ví dụ: Một đứa trẻ thấy một con khỉ ăn chuối. Nhờ biết rằng "Khỉ là một loài động vật", nó suy diễn ngay "Động vật cần ăn để sống" chứ không cần phải nhìn thấy 10.000 con khỉ ăn chuối khác nhau.

◇ **Bài toán:** Nhúng Tri thức nền vào hệ thống AI để thúc đẩy quá trình học quy nạp.

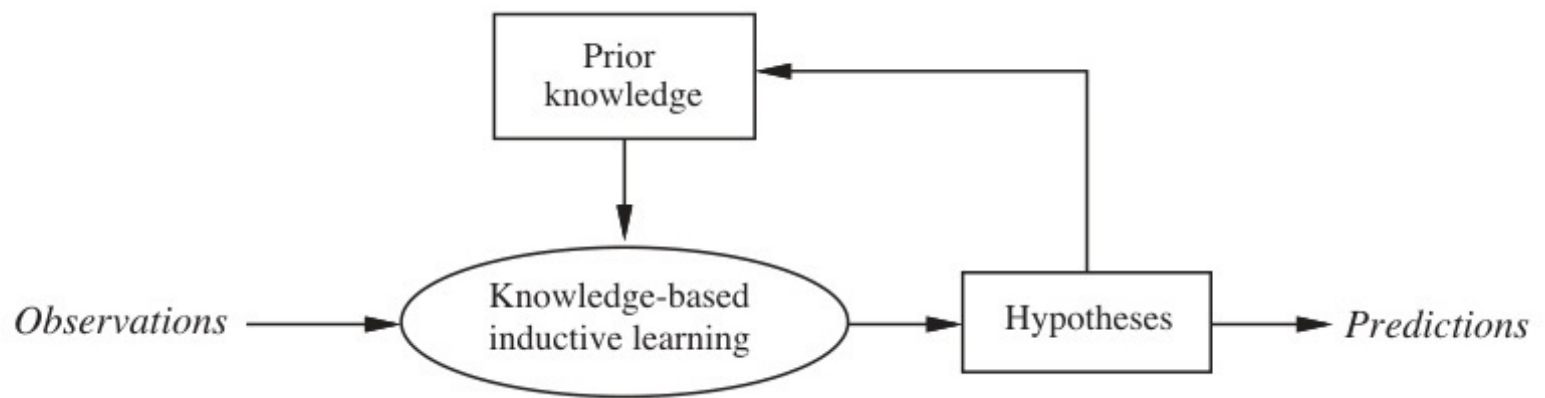


Figure 3: Một tác nhân học tập tích lũy liên tục cập nhật kho tri thức nền của nó theo thời gian.

20.2 Lược đồ Diễn dịch & Quy nạp

◇ Tri thức nền *Background* và Lệnh phân loại *Classifications* được gắn kết như thế nào?

◇ **Học dựa trên diễn dịch (Deductive Learning):**

- $Background \wedge Hypothesis \wedge Descriptions \models Classifications$
- Từ tri thức có sẵn, ta suy diễn ra kết luận logic mà không cần nhiều phép tính dự đoán thống kê.

◇ **Học quy nạp (Inductive Learning):**

- Từ một tập hợp các ví dụ ($Descriptions + Classifications$) và Tri thức nền, ta "Đảo ngược" logic để tìm ra quy luật ($Hypothesis$).

20.3 Học tập Dựa trên Giải thích (EBL)

◇ **Explanation-Based Learning (EBL):**

- Một cơ chế giúp hệ thống học cách **tăng tốc** giải quyết bài toán.
- Nó không tạo ra tri thức "Mới" về mặt nguyên lý, mà nó chuyển hóa tri thức tiềm ẩn thành dạng dễ sử dụng.

◇ Ví dụ: Đạo hàm hàm số $x^2 \sin(x)$.

- Máy tính phải áp dụng quy tắc chuỗi, quy tắc tích rất vất vả.
- Sau lần giải đầu tiên, EBL sẽ "Lưu" (Cache) lại quy tắc tắt: $(f(x)g(x))' = f'g + fg'$ để lần sau gặp lại bài toán tương tự thì giải ngay trong 1 bước.

20.3 Quá trình Trích xuất trong EBL

◇ ****Các bước của thuật toán EBL:****

1. **Xây dựng Cây chứng minh (Proof Tree):** Giải quyết bài toán bằng cách sử dụng các quy tắc suy diễn cơ bản.
2. **Phân tích chứng minh:** Rút ra nguyên lý cốt lõi, loại bỏ chi tiết ngẫu nhiên.
3. **Khái quát hóa (Generalization):** Tạo ra một luật mới ở cấp độ trừu tượng.
4. **Cập nhật (Update):** Thêm luật mới vào cơ sở tri thức để tương lai chạy nhanh hơn.

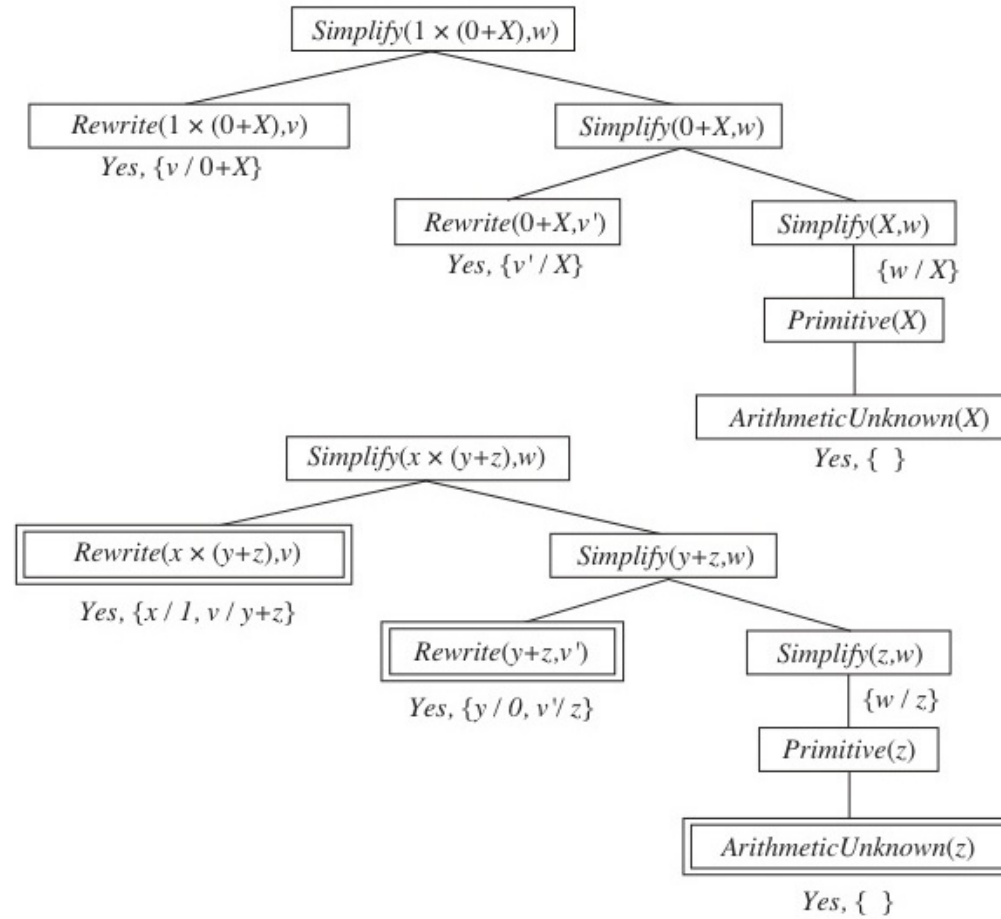


Figure 4: Cây chứng minh cho bài toán rút gọn (EBL) nhằm tối ưu hóa quá trình giải đáp.

20.3 Sự đánh đổi hiệu suất (Utility Problem)

- ◇ Lợi ích của EBL là rất lớn, nhưng nó cũng có điểm yếu: **Utility Problem**.
- ◇ Nếu ta cứ tiếp tục lưu trữ (memoize) mọi quy tắc tắt (shortcuts) mà ta từng gặp, Cơ sở tri thức (Knowledge Base) sẽ phình to ra hàng triệu luật.
- ◇ **Hậu quả:** Việc tra cứu xem nên áp dụng luật nào trong 1 triệu luật đó sẽ tốn *Nhiều thời gian hơn* so với việc giải bài toán từ đầu.
- ◇ **Cách khắc phục:** Hệ thống EBL chỉ nên lưu trữ các quy tắc được sử dụng thường xuyên (High probability of reuse) và có chi phí diễn dịch cao.

20.4 Học sử dụng Thông tin Tương quan

◇ Một ứng dụng khác của Tri thức nền là thu hẹp **Không gian giả thuyết (Hypothesis Space)**.

◇ **Thông tin tương quan (Relevance Information):**

- Giúp AI biết rằng: Thuộc tính này là quan trọng, còn thuộc tính kia là vô nghĩa.
- Ví dụ dự đoán quốc tịch của một người: "Ngôn ngữ mẹ đẻ" là có liên quan (Relevant), còn "Màu áo đang mặc" là không liên quan (Irrelevant).

◇ Bằng cách loại bỏ các thuộc tính không liên quan ngay từ đầu nhờ Tri thức nền, AI tiết kiệm được khối lượng tính toán theo cấp số nhân.

20.5 Lập trình Logic Quy nạp (ILP)

◇ Học máy truyền thống (như Cây Quyết Định) rất hạn chế ở khả năng biểu diễn.

◇ **Inductive Logic Programming (ILP):**

- Khởi nguồn từ ngôn ngữ Prolog.
- Mục tiêu: Thay vì sinh ra một hàm số vô hình, ILP sinh ra một **Chương trình Logic (A Logic Program)**.
- ILP có thể học được các *Khái niệm Quan hệ (Relational Concepts)* và *Đệ quy (Recursive)*.

20.5 Điểm vượt trội của ILP so với Cây Quyết định

◇ Bài toán Quan hệ gia đình (Family Tree):

- Cho dữ liệu: $Cha(John, Mary)$, $Cha(David, John)$.
- Hãy dự đoán quan hệ $OngNoi(x, y)$.

◇ **Cây quyết định:** Không thể biểu diễn nổi vì không có cách nối các biến.

◇ **ILP:** Chỉ cần 1 ví dụ, nó có thể tự học quy luật đệ quy đẹp mắt:

- $OngNoi(x, y) \Leftarrow Cha(x, z) \wedge Cha(z, y)$

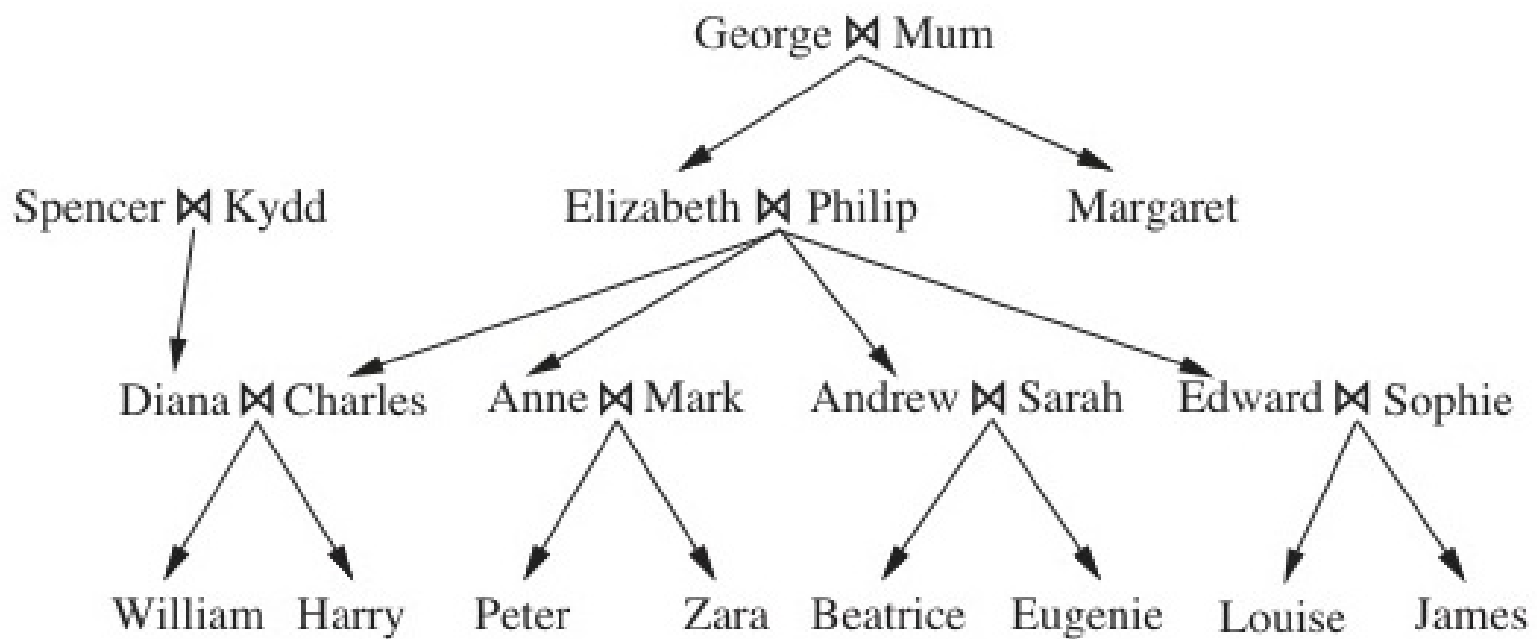


Figure 5: Một cây gia phả điển hình thể hiện tính chất quan hệ mà Lập trình Logic Quy nạp (ILP) dễ dàng giải quyết.

20.5 Học quy nạp từ trên xuống (Top-Down)

◇ Cách ILP sinh ra giả thuyết: Bắt đầu từ quy tắc tổng quát nhất rồi đi vào chi tiết.

◇ **Thuật toán FOIL (First-Order Inductive Learner):**

- Giống như học Cây quyết định nhưng áp dụng trên Mệnh đề Horn (Horn clauses).
- Nó liên tục thêm các điều kiện logic (literals) $\wedge$ vào mệnh đề cho đến khi không còn ví dụ Âm tính (Negative examples) nào khớp với mệnh đề đó.
- Nó dùng Mức tăng Thông tin (Information Gain) để quyết định nên chọn điều kiện logic nào.

```

function FOIL(examples, target) returns a set of Horn clauses
  inputs: examples, set of examples
           target, a literal for the goal predicate
  local variables: clauses, set of clauses, initially empty

  while examples contains positive examples do
    clause ← NEW-CLAUSE(examples, target)
    remove positive examples covered by clause from examples
    add clause to clauses
  return clauses

```

```

function NEW-CLAUSE(examples, target) returns a Horn clause
  local variables: clause, a clause with target as head and an empty body
                   l, a literal to be added to the clause
                   extended_examples, a set of examples with values for new variables

  extended_examples ← examples
  while extended_examples contains negative examples do
    l ← CHOOSE-LITERAL(NEW-LITERALS(clause), extended_examples)
    append l to the body of clause
    extended_examples ← set of examples created by applying EXTEND-EXAMPLE
                        to each example in extended_examples
  return clause

```

```

function EXTEND-EXAMPLE(example, literal) returns a set of examples
  if example satisfies literal
    then return the set of examples created by extending example with
                each possible constant value for each new variable in literal
  else return the empty set

```

Figure 6: Phác thảo thuật toán FOIL dùng để học các tập hợp mệnh đề Horn bậc nhất từ các ví dụ.

20.5 Học quy nạp với Diễn dịch đảo ngược

◇ Diễn dịch đảo ngược (Inverse Deduction):

- Trong Logic thông thường (Deduction): $A \wedge (A \Rightarrow B) \vdash B$. (Cho trước nguyên nhân, chứng minh kết quả).
- Trong ILP (Induction): Nếu biết kết quả B và $A \Rightarrow B$, ta phải "Đảo ngược" chứng minh để đi tìm Nguyên nhân A .

◇ Đây là một bước đột phá của Toán học: Cho phép AI "Phát hiện tri thức mới" thay vì chỉ nội suy thông tin.

◇ **Ứng dụng cực khủng của ILP:** Thiết kế cấu trúc phân tử thuốc, tìm quy luật gập Protein (Protein folding) trong sinh học phân tử, điều kiện mà Deep Learning trước đây phải chật vật.

167

Figure 7: Các bước ban đầu trong một quá trình phân giải đảo ngược (Inverse Deduction).

Tóm tắt Chương 20

- ◇ **Học Máy Không Giám Sát và Có Giám Sát thuần túy** bị giới hạn bởi sự thiếu hụt kiến thức phổ thông.
- ◇ **Tri thức Nền (Background Knowledge)** đóng vai trò như một bàn đạp để AI nhảy cóc qua hàng nghìn ví dụ vô nghĩa.
- ◇ **EBL (Học qua Giải thích):** Kỹ thuật tối ưu hóa nhằm tăng tốc suy luận dựa trên việc rút gọn các bài toán đã giải.
- ◇ **Lập trình Logic Quy nạp (ILP):** Biểu tượng rực rỡ nhất của sự kết hợp giữa Khả năng tự học (Learning) và Khả năng lập luận sâu sắc (Reasoning) bằng logic bậc nhất.